



L OVELY
P ROFESSIONAL
U NIVERSITY

Transforming Education Transforming India

School of Computer Science and Engineering

Topic – Public Safety Data Analysis using Linear Regression

Submitted by -

Name – Hari Om Singh

Registration Number – 12211418

Roll No – B43

Section – K22ZT

Course – Cluster Computing (INT315)

Submitted to -

Name of faculty – Zeenat Zahra (30447)

Introduction

Public safety is one of the most critical aspects of urban management, directly influencing the quality of life and overall community well-being. With the increasing volume of data generated by law enforcement agencies, predictive analytics has become an essential tool for understanding crime trends and improving decision-making. This project — *Public Safety Data Analysis using Linear Regression* — aims to apply data-driven methods to uncover meaningful patterns in crime data and support proactive safety measures.

In this study, an open-source public safety dataset is analyzed to explore how various factors such as city, time, and victim demographics relate to the number of reported incidents. The project leverages **Linear Regression**, a fundamental machine learning algorithm, to model and predict crime occurrences. By quantifying relationships between independent variables and the number of incidents, the model provides insights that can help authorities allocate resources more efficiently, identify high-risk areas, and anticipate potential crime surges.

The workflow includes several key stages: data acquisition, preprocessing and encoding of categorical variables, model training and evaluation, and visualization of results. Evaluation metrics such as Root Mean Squared Error (RMSE) and R^2 score are used to assess model accuracy. Finally, the project concludes with interpretations of the findings and their practical implications for enhancing public safety through predictive analytics.

Source Code

```
scala> import org.apache.spark.sql.Session
```

```
import org.apache.spark.sql.Session
```

```
scala> import org.apache.spark.sql.functions._
```

```
import org.apache.spark.sql.functions._
```

```
scala> import org.apache.spark.ml.feature.{StringIndexer, OneHotEncoder, VectorAssembler}
```

```
import org.apache.spark.ml.feature.{StringIndexer, OneHotEncoder, VectorAssembler}
```

```
scala> import org.apache.spark.ml.regression.LinearRegression
```

```
import org.apache.spark.ml.regression.LinearRegression
```

```
scala> import org.apache.spark.ml.Pipeline
```

```
import org.apache.spark.ml.Pipeline
```

```
scala> import org.apache.spark.ml.evaluation.RegressionEvaluator
```

```
import org.apache.spark.ml.evaluation.RegressionEvaluator
```

```
scala> val spark = SparkSession.builder.appName("Proj40-LR").master("local[*]").getOrCreate()
```

```
warning: 1 deprecation (since 2.13.3); for details, enable `:setting -deprecation` or `:replay -deprecation`
```

```
25/11/12 18:54:45 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
```

```
val spark: org.apache.spark.sql.SparkSession = org.apache.spark.sql.classic.SparkSession@671e3bd4
```

```
scala> val raw =
```

```
spark.read.option("header","true").option("inferSchema","true").csv("C:/Users/hp/Downloads/crime_safety_dataset.csv")
```

^

```
error: ')' expected but '.' found.
```

```
scala> val raw =
```

```
spark.read.option("header","true").option("inferSchema","true").csv("C:/Users/hp/Downloads/crime_safety_dataset.csv")
```

```
val raw: org.apache.spark.sql.DataFrame = [id: int, date: date ... 8 more fields]
```

```
scala> val dfDate = raw.withColumn("date_parsed", to_date($"date", "yyyy-MM-dd")).withColumn("year", year($"date_parsed"))
```

```
).withColumn("month", month($"date_parsed"))
```

```
val dfDate: org.apache.spark.sql.DataFrame = [id: int, date: date ... 11 more fields]
```

```
scala> val monthly =
```

```
dfDate.groupBy("city","year","month").agg(count("*").alias("number_of_incidents"),avg("victim_age"))
```

```
.alias("avg_victim_age"),countDistinct("crime_type").alias("crime_variety")).na.drop()
```

```
val monthly: org.apache.spark.sql.DataFrame = [city: string, year: int ... 4 more fields]
```

```
scala> val cityIdx = new StringIndexer().setInputCol("city").setOutputCol("city_idx").setHandleInvalid("keep")
```

```
val cityIdx: org.apache.spark.ml.feature.StringIndexer = strIdx_25748040d030
```

```
scala> val cityOhe = new OneHotEncoder().setInputCols(Array("city_idx")).setOutputCols(Array("city_vec"))
val cityOhe: org.apache.spark.ml.feature.OneHotEncoder = oneHotEncoder_dfae4ec5759e
```

```
scala> val assembler = new
VectorAssembler().setInputCols(Array("city_vec", "avg_victim_age", "crime_variety", "month", "year")).setOutputCol("features")
```

```
val assembler: org.apache.spark.ml.feature.VectorAssembler = VectorAssembler:
uid=vecAssembler_f7a652a7dff2, handleInvalid=error, numInputCols=5
```

```
scala> val lr = new
LinearRegression().setLabelCol("number_of_incidents").setFeaturesCol("features").setMaxIter(100).setRegParam(0.1)
```

```
val lr: org.apache.spark.ml.regression.LinearRegression = linReg_ee0756cfe29f
```

```
scala> val pipeline = new Pipeline().setStages(Array(cityIdx, cityOhe, assembler, lr))
```

```
val pipeline: org.apache.spark.ml.Pipeline = pipeline_b6c5e1c19b07
```

```
scala> val Array(train, test) = monthly.randomSplit(Array(0.8, 0.2), seed = 42)
```

```
val train: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [city: string, year: int ... 4 more fields]
```

```
val test: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [city: string, year: int ... 4 more fields]
```

```
scala> val model = pipeline.fit(train)
```

```
25/11/12 18:58:46 WARN InstanceBuilder: Failed to load implementation
from:dev.ludovic.netlib.lapack.JNILAPACK
```

```
val model: org.apache.spark.ml.PipelineModel = pipeline_b6c5e1c19b07
```

```
scala> val preds = model.transform(test).cache()
```

```
val preds: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [city: string, year: int ... 8 more fields]
```

```
scala> preds.select("city", "year", "month", "number_of_incidents", "prediction").show(10, false)
```

```
+-----+-----+-----+-----+-----+
|city    |year|month|number_of_incidents|prediction    |
+-----+-----+-----+-----+-----+
|Dallas  |2021|8    |1                |1.223254076607784 |
|San Antonio|2025|6    |2                |1.1259193895161488|
```

Philadelphia	2025	2	1	1.048832556084772
Houston	2023	10	2	2.007635115010679
San Jose	2022	11	1	1.1702582593904438
Chicago	2021	8	3	3.1722220726985206
Houston	2020	12	3	3.079775922846899
San Diego	2023	2	2	2.0443749259159887
New York	2020	9	1	1.1262268213470072
San Jose	2023	12	2	2.116259064245302

+-----+-----+-----+-----+

only showing top 10 rows

```
scala> val eval = new
RegressionEvaluator().setLabelCol("number_of_incidents").setPredictionCol("prediction")
```

```
val eval: org.apache.spark.ml.evaluation.RegressionEvaluator = RegressionEvaluator:
uid=regEval_549e68c9b3e8, metricName=rmse, throughOrigin=false
```

```
scala> println("RMSE = " + eval.setMetricName("rmse").evaluate(preds))
```

```
RMSE = 0.3112713663342451
```

```
scala> println("MAE = " + eval.setMetricName("mae").evaluate(preds))
```

```
MAE = 0.1986932108185975
```

```
scala> println("R2 = " + eval.setMetricName("r2").evaluate(preds))
```

```
R2 = 0.9265911763135026
```

Output

```
scala> import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.SparkSession

scala> import org.apache.spark.sql.functions._
import org.apache.spark.sql.functions._

scala> import org.apache.spark.ml.feature.{StringIndexer, OneHotEncoder, VectorAssembler}
import org.apache.spark.ml.feature.{StringIndexer, OneHotEncoder, VectorAssembler}

scala> import org.apache.spark.ml.regression.LinearRegression
import org.apache.spark.ml.regression.LinearRegression

scala> import org.apache.spark.ml.Pipeline
import org.apache.spark.ml.Pipeline

scala> import org.apache.spark.ml.evaluation.RegressionEvaluator
import org.apache.spark.ml.evaluation.RegressionEvaluator

scala> val spark = SparkSession.builder.appName("Proj40-LR").master("local[*]").getOrCreate()
warning: 1 deprecation (since 2.13.3); for details, enable `:setting -deprecation` or `:replay -deprecation`
25/11/12 18:54:45 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
val spark: org.apache.spark.sql.SparkSession = org.apache.spark.sql.classic.SparkSession@671e3bd4

scala> val raw = spark.read.option("header","true").option("inferSchema","true").csv("C:/Users/hp/Downloads/crime_safety_dataset.csv")

error: ')' expected but '.' found.

scala> val raw = spark.read.option("header","true").option("inferSchema","true").csv("C:/Users/hp/Downloads/crime_safety_dataset.csv")
val raw: org.apache.spark.sql.DataFrame = [id: int, date: date ... 8 more fields]

scala> val dfDate = raw.withColumn("date_parsed", to_date($"date", "yyyy-MM-dd")).withColumn("year", year($"date_parsed"))
.withColumn("month", month($"date_parsed"))
val dfDate: org.apache.spark.sql.DataFrame = [id: int, date: date ... 11 more fields]

scala> val monthly = dfDate.groupBy("city","year","month").agg(count("*").alias("number_of_incidents"),avg("victim_age").alias("avg_victim_age"),countDistinct("crime_type").alias("crime_variety")).na.drop()
val monthly: org.apache.spark.sql.DataFrame = [city: string, year: int ... 4 more fields]

scala> val cityIdx = new StringIndexer().setInputCol("city").setOutputCol("city_idx").setHandleInvalid("keep")
val cityIdx: org.apache.spark.ml.feature.StringIndexer = strIdx_2574804d030

scala> val cityOhe = new OneHotEncoder().setInputCols(Array("city_idx")).setOutputCols(Array("city_vec"))
val cityOhe: org.apache.spark.ml.feature.OneHotEncoder = oneHotEncoder_df4e4ec5759e

scala> val assembler = new VectorAssembler().setInputCols(Array("city_vec","avg_victim_age","crime_variety","month","year")).setOutputCol("features")
```

```
scala> val assembler = new VectorAssembler().setInputCols(Array("city_vec","avg_victim_age","crime_variety","month","year")).setOutputCol("features")
val assembler: org.apache.spark.ml.feature.VectorAssembler = VectorAssembler: uid=vecAssembler_f7a652a7dff2, handleInvalid=error, numInputCols=5

scala> val lr = new LinearRegression().setLabelCol("number_of_incidents").setFeaturesCol("features").setMaxIter(100).setRegParam(0.1)
val lr: org.apache.spark.ml.regression.LinearRegression = linReg_ee0756cfe29f

scala> val pipeline = new Pipeline().setStages(Array(cityIdx, cityOhe, assembler, lr))
val pipeline: org.apache.spark.ml.Pipeline = pipeline_b6c5e1c19b07

scala> val Array(train, test) = monthly.randomSplit(Array(0.8,0.2), seed = 42)
val train: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [city: string, year: int ... 4 more fields]
val test: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [city: string, year: int ... 4 more fields]

scala> val model = pipeline.fit(train)
25/11/12 18:58:46 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.lapack.JNILAPACK
val model: org.apache.spark.ml.PipelineModel = pipeline_b6c5e1c19b07

scala> val preds = model.transform(test).cache()
val preds: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [city: string, year: int ... 8 more fields]

scala> preds.select("city","year","month","number_of_incidents","prediction").show(10,false)
+-----+-----+-----+-----+-----+
|city      |year|month|number_of_incidents|prediction|
+-----+-----+-----+-----+-----+
|Dallas    |2021|8    |1                |1.223254076607784|
|San Antonio|2025|6    |2                |1.1259193895161488|
|Philadelphia|2025|2    |1                |1.048832556084772|
|Houston   |2023|10   |2                |2.007635115010679|
|San Jose  |2022|11   |1                |1.1702582593904438|
|Chicago   |2021|8    |3                |3.1722220726985206|
|Houston   |2020|12   |3                |3.079775922846899|
|San Diego |2023|2    |2                |2.0443749259159887|
|New York  |2020|9    |1                |1.1262268213470072|
|San Jose  |2023|12   |2                |2.116259064245302|
+-----+-----+-----+-----+-----+
only showing top 10 rows
```

```
scala> preds.select("city","year","month","number_of_incidents","prediction").show(10,false)
+-----+-----+-----+-----+-----+
|city      |year|month|number_of_incidents|prediction|
+-----+-----+-----+-----+-----+
|Dallas    |2021|8     |1                 |1.223254076607784|
|San Antonio|2025|6     |2                 |1.1259193895161488|
|Philadelphia|2025|2     |1                 |1.048832556084772|
|Houston    |2023|10    |2                 |2.007635115010679|
|San Jose   |2022|11    |1                 |1.1702582593904438|
|Chicago    |2021|8     |3                 |3.1722220726985206|
|Houston    |2020|12    |3                 |3.079775922846899|
|San Diego  |2023|2     |2                 |2.0443749259159887|
|New York   |2020|9     |1                 |1.1262268213470072|
|San Jose   |2023|12    |2                 |2.116259064245302|
+-----+-----+-----+-----+-----+
only showing top 10 rows

scala> val eval = new RegressionEvaluator().setLabelCol("number_of_incidents").setPredictionCol("prediction")
val eval: org.apache.spark.ml.evaluation.RegressionEvaluator = RegressionEvaluator: uid=regEval_549e68c9b3e8, metricName=rmse, throughOrigin=false

scala> println("RMSE = " + eval.setMetricName("rmse").evaluate(preds))
RMSE = 0.3112713663342451

scala> println("MAE = " + eval.setMetricName("mae").evaluate(preds))
MAE = 0.1986932108185975

scala> println("R2 = " + eval.setMetricName("r2").evaluate(preds))
R2 = 0.9265911763135026
```

Dashboard

